

Arquitecturas Distribuidas en la Industria

Patron analizado: Microservicios | Empresa: Netflix

1) Descripcion del caso

Netflix, presente en más de 190 países, migró entre 2009 y 2012 de un monolito centralizado hacia una arquitectura de microservicios en Amazon Web Services (AWS). Cada función autenticación, recomendaciones, transcodificación, facturación opera como servicio independiente con su propia base de datos y ciclo de despliegue. Hoy la plataforma supera los 1000 microservicios que colaboran mediante APIs REST y colas de eventos [2].

2) Por que eligieron microservicios

El monolito no permitía escalar componentes por separado: el motor de recomendaciones y el transcodificador requiere recursos distintos. Además, cualquier fallo derribaba el sistema completo y redesplegar un cambio menor exigía recompilar toda la aplicación. Los microservicios resolvieron los tres problemas: escalabilidad diferenciada, aislamiento de fallos mediante *circuit breaker*, y despliegues independientes por equipos autónomos [1].

3) Beneficios obtenidos

Netflix alcanzo disponibilidad del 99,99 %, recuperandose de fallos parciales sin impacto para el usuario. El escalado por servicio redujo costos al asignar cómputo solo donde habia demanda. La velocidad de innovacion aumento al permitir despliegues múltiples por dia, por equipo. Como subproducto, herramientas internas como Hystrix (circuit breaker) y Zuul (API Gateway) fueron liberadas como código abierto y adoptadas globalmente [1].

4) Desafios encontrados

La coordinación de más de 1000 servicios requirió una inversión constante en trazabilidad distribuida y orquestación de contenedores. La necesidad de mantener la consistencia entre los servicios llevó a la adopción de consistencia eventual a través de eventos asincrónicos. La latencia en la red provocó errores que no existían en el monolito, lo que impulsó *Chaos Engineering* —**Chaos Monkey**— para construir resiliencia por diseño. Blinowski et al. advierten que para sistemas de menor escala estos costos pueden superar los beneficios [1].

5) Aplicacion al proyecto propio

El SGA Provincias Unidas aplica los mismos principios a escala institucional. La separación en MS Principal y MS Docente garantiza disponibilidad diferenciada: Si el módulo docente falla, secretaría continúa operando. RabbitMQ replica el patrón de eventos asincrónicos de Netflix para sincronizar estados sin acoplamiento. El API Gateway con Spring Cloud Gateway cumple la función de Zuul: Entrada única con JWT y rate limiting. Con 344 estudiantes y 20 usuarios concurrentes, la arquitectura se dimensiona proporcionalmente al contexto real [2].

Referencias

- [1] Blinowski, G., Ojdowska, A., y Przybylek, A. (2022). *Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation*. *IEEE Access*, 10, 20357–20374. DOI: <https://doi.org/10.1109/ACCESS.2022.3152803> — PDF: <https://ieeexplore.ieee.org/document/9717259>
- [2] Dragoni, N., Giallorenzo, S., Lafuente, A. L., et al. (2017). *Microservices: Yesterday, Today, and Tomorrow*. En *Present and Ulterior Software Engineering*, Springer, pp. 195–216. DOI: https://doi.org/10.1007/978-3-319-67425-4_12 — PDF: <https://arxiv.org/pdf/1606.04036>